

Algorithms 2

Metric Traveling Salesperson Problem

Daniel Rosenberg & Michael Trushkin

Ariel University

1. The Metric TSP

- n cities denoted by $[n]$
- **Symmetric** cost matrix $C := (c_{ij} \geq 0)$:
 - $c_{ii} = 0$ for all $i \in [n]$
 - **$\triangle$ -inequality**: $c_{ij} + c_{jk} \geq c_{ik}$ for all i, j, k

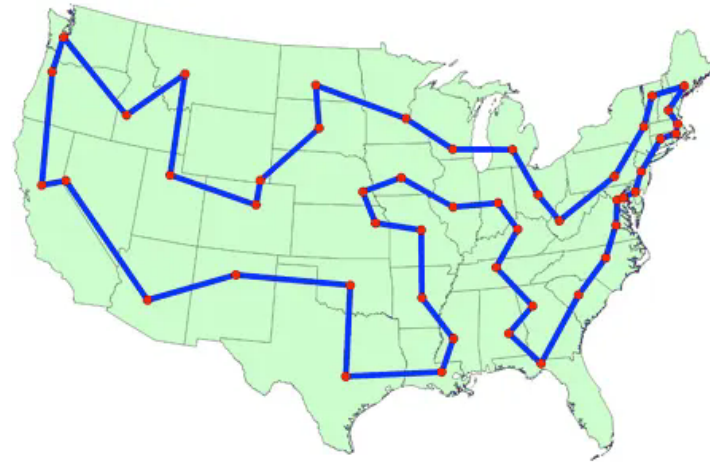


Figure 1: A TSP tour through US cities

- n cities denoted by $[n]$
- **Symmetric** cost matrix $C := (c_{ij} \geq 0)$:
 - $c_{ii} = 0$ for all $i \in [n]$
 - **$\triangle$ -inequality**: $c_{ij} + c_{jk} \geq c_{ik}$ for all i, j, k

We view C as a complete **edge-weighted** graph on n vertices.

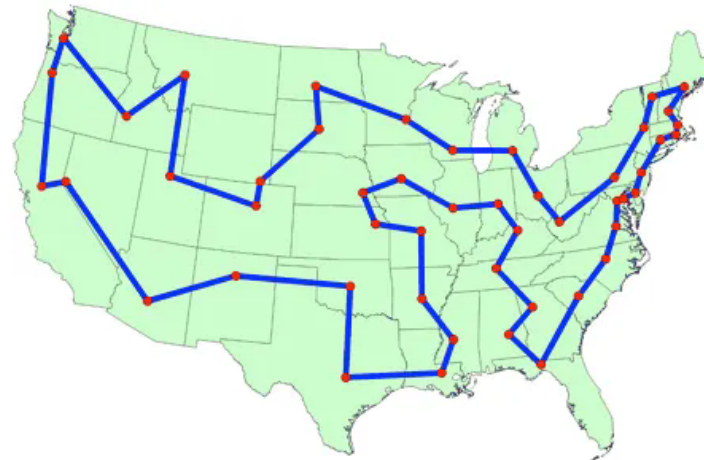


Figure 1: A TSP tour through US cities

- n cities denoted by $[n]$
- **Symmetric** cost matrix $C := (c_{ij} \geq 0)$:
 - $c_{ii} = 0$ for all $i \in [n]$
 - **$\triangle$ -inequality**: $c_{ij} + c_{jk} \geq c_{ik}$ for all i, j, k

We view C as a complete **edge-weighted** graph on n vertices.

Problem 3 Metric TSP ($\triangle$ TSP)

Find: a minimum cost **Hamiltonian cycle** (visit every city exactly once and return).

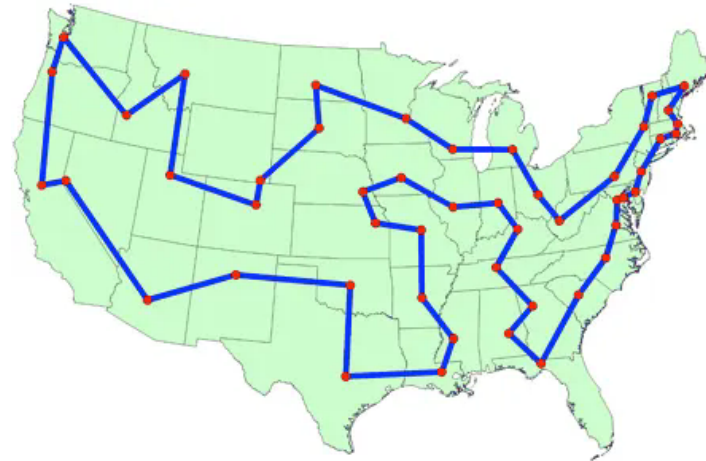


Figure 1: A TSP tour through US cities

Observation 1 Removing **any single edge** from a TSP-tour leaves a **spanning tree**.

Observation 2 Removing **any single edge** from a TSP-tour leaves a **spanning tree**.

Why? A Hamiltonian cycle visits all n vertices and uses n edges. Remove one edge $\rightarrow$ a path visiting all n vertices $\rightarrow$ a **spanning tree** (connected, $n - 1$ edges, no cycles).

Observation 3 Removing **any single edge** from a TSP-tour leaves a **spanning tree**.

Why? A Hamiltonian cycle visits all n vertices and uses n edges. Remove one edge $\rightarrow$ a path visiting all n vertices $\rightarrow$ a **spanning tree** (connected, $n - 1$ edges, no cycles).

Theorem 3 Cost of optimal TSP-tour $\geq$ Cost of a minimum spanning tree (MST)

Observation 4 Removing **any single edge** from a TSP-tour leaves a **spanning tree**.

Why? A Hamiltonian cycle visits all n vertices and uses n edges. Remove one edge $\rightarrow$ a path visiting all n vertices $\rightarrow$ a **spanning tree** (connected, $n - 1$ edges, no cycles).

Theorem 4 Cost of optimal TSP-tour $\geq$ Cost of a minimum spanning tree (MST)

Proof. Let T^* be an optimal tour and e any edge of T^* . Then $T^* - e$ is a spanning tree, so

$$\text{MST cost} \leq c(T^* - e) < c(T^*) = \text{OPT}. \quad \square$$

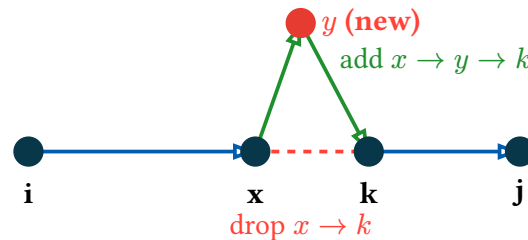
2. Nearest Extension Algorithm

Algorithm 1: Nearest Extension

```

1: Input: cities  $[n]$ , costs  $C$ 
2: Let  $i, j \in [n]$  be the two closest cities
3:  $T \leftarrow (\{i, j\}, \{i \rightarrow j\})$ 
4: while  $V(T) \neq [n]$  do
5:    $(x, y) \leftarrow \arg \min \{c_{xy} : x \in V(T), y \notin V(T)\}$ 
6:   Let  $k =$  vertex after  $x$  along  $T$ 
7:    $T \leftarrow T - (x \rightarrow k) + (x \rightarrow y) + (y \rightarrow k)$ 
8: end
9: return  $T + (j \rightarrow i)$ 
    
```

Idea: at each step, **insert** the nearest unvisited city y into the tour right after its closest tour-vertex x .



Theorem 5 The Nearest Extension algorithm is a **2-approximation** for $\triangle$ TSP.

Theorem 6 The Nearest Extension algorithm is a **2-approximation** for $\triangle$ TSP.

Let F' = edges chosen greedily (one per extension step), and $F := F' \cup \{ij\}$.

Theorem 7 The Nearest Extension algorithm is a **2-approximation** for $\triangle$ TSP.

Let F' = edges chosen greedily (one per extension step), and $F := F' \cup \{ij\}$.

Claim : $([n], F)$ is a **minimum spanning tree**.

- Each edge of F' is the **cheapest edge crossing some cut** (greedy choice);
- same for ij (closest pair).
- So $|F'| = n - 2$ extension steps $\Rightarrow |F| = n - 1$, and there is an execution of **Prim's algorithm** that produces F .
 $\square$

Theorem 8 The Nearest Extension algorithm is a **2-approximation** for $\triangle$ TSP.

Let F' = edges chosen greedily (one per extension step), and $F := F' \cup \{ij\}$.

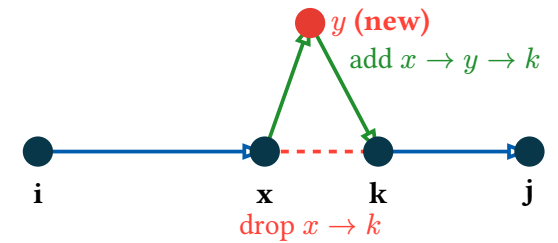
Claim : $([n], F)$ is a **minimum spanning tree**.

- Each edge of F' is the **cheapest edge crossing some cut** (greedy choice);
- same for ij (closest pair).
- So $|F'| = n - 2$ extension steps $\Rightarrow |F| = n - 1$, and there is an execution of **Prim's algorithm** that produces F .
 $\square$

Hence by Theorem 4

$$\text{OPT} \geq c(F) = \text{weight of MST.}$$

The returned tour $T + ij$ is obtained by a series of **bypasses**: each extension replaces $f := (x \rightarrow k)$ with $(x \rightarrow y \rightarrow k)$.



The returned tour $T + ij$ is obtained by a series of **bypasses**: each extension replaces $f := (x \rightarrow k)$ with $(x \rightarrow y \rightarrow k)$.

For each bypass, the **cost increase** is:

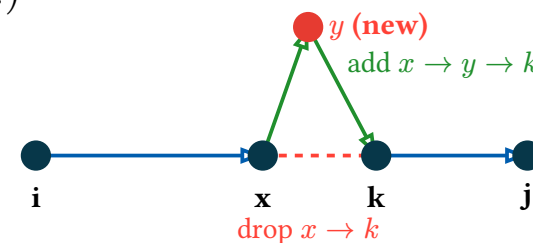
$$\underbrace{c_{xy} + c_{yk}}_{\text{cost of new edges}} - \underbrace{c_{xk}}_{\text{cost of old edge}} \quad (\text{non-negative by } \triangle \text{-ineq.})$$

By the $\triangle$ -inequality we have

$$c_{yk} \leq c_{yx} + c_{xk}$$

So that:

$$\text{increase} = c_{xy} + c_{yk} - c_{xk} \leq c_{xy} + (c_{yx} + c_{xk}) - c_{xk} \leq 2c_{xy} = 2c(f). \quad (\text{since } xy \in F')$$



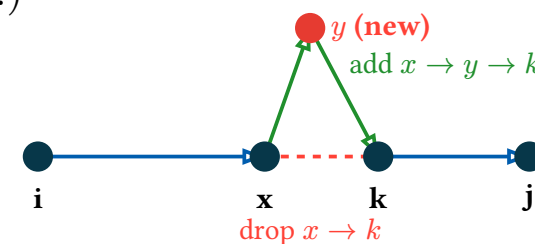
The returned tour $T + ij$ is obtained by a series of **bypasses**: each extension replaces $f := (x \rightarrow k)$ with $(x \rightarrow y \rightarrow k)$.

For each bypass, the **cost increase** is:

$$\underbrace{c_{xy} + c_{yk}}_{\text{cost of new edges}} - \underbrace{c_{xk}}_{\text{cost of old edge}} \quad (\text{non-negative by } \triangle \text{-ineq.})$$

By the $\triangle$ -inequality we have

$$c_{yk} \leq c_{yx} + c_{xk}$$



So that:

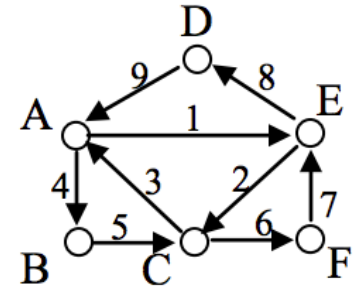
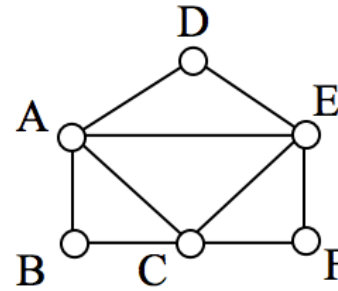
$$\text{increase} = c_{xy} + c_{yk} - c_{xk} \leq c_{xy} + (c_{yx} + c_{xk}) - c_{xk} \leq 2c_{xy} = 2c(f). \quad (\text{since } xy \in F')$$

Summing over all bypasses plus the final closing edge ji :

$$c(T + ij) \leq c_{ij} + \sum_{f \in F'} 2c(f) \leq 2 \sum_{f \in F} c(f) \leq 2 \cdot \text{OPT}. \quad \square$$

3. Doubling Trees Algorithm

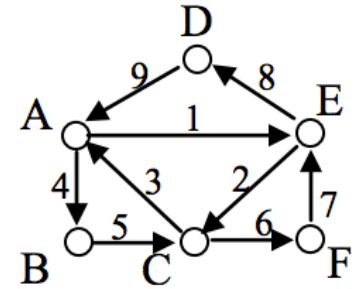
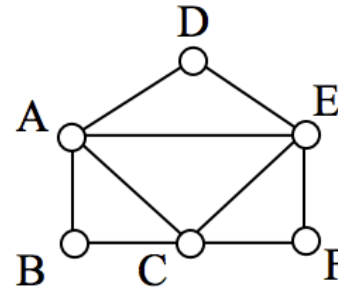
Definition 8 An **Euler tour** of a connected multigraph is a **closed walk** that traverses every edge **exactly once**.



Definition 9 An **Euler tour** of a connected multigraph is a **closed walk** that traverses every edge **exactly once**.

Theorem 10 G -multigraph **Eulerian** $\Leftrightarrow$ every vertex has **even degree**.

 **Remark** Finding an Euler tour is in P .



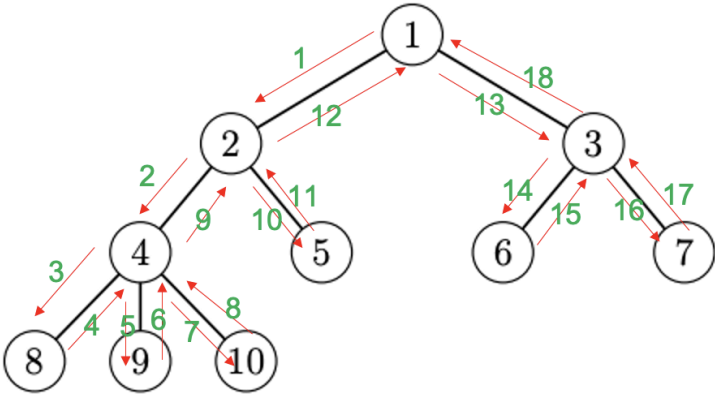
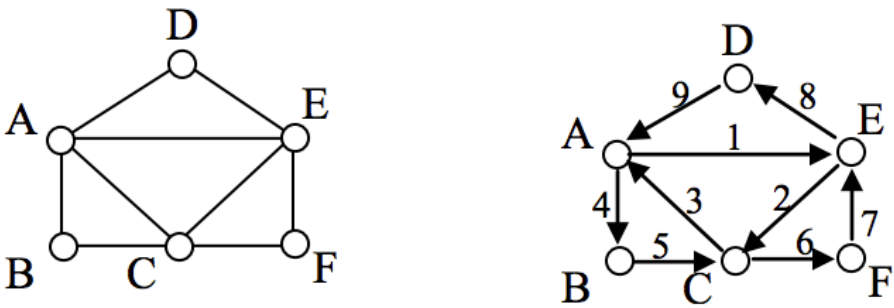
Definition 10 An **Euler tour** of a connected multigraph is a **closed walk** that traverses every edge **exactly once**.

Theorem 11 G -multigraph **Eulerian** $\Leftrightarrow$ every vertex has **even degree**.

Remark Finding an Euler tour is in P .

Observation 7 If T is a tree, **doubling** every edge gives an Eulerian multigraph.

Proof: Every vertex gets degree $2 \cdot \deg_{T(v)}$ — always even. $\square$



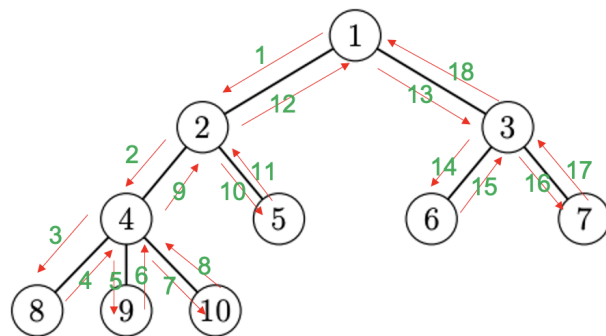
Euler tour:

1	2	4	8	4	9	4	10	4	2	5	2	1	3	6	3	7	3	1
---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---

Algorithm 2: Doubling Trees

- 1: **Input:** complete graph G with costs C
- 2: $T \leftarrow$ MST of G
- 3: $T' \leftarrow$ double every edge of T
- 4: $\mathcal{E} \leftarrow$ Euler tour of T'
- 5: **return** TSP-tour from $\mathcal{E}$ by **shortcutting**

Shortcutting: traverse $\mathcal{E}$; keep only the **first appearance** of each vertex. By the $\triangle$ -inequality, shortcuts never increase cost.



Euler tour:

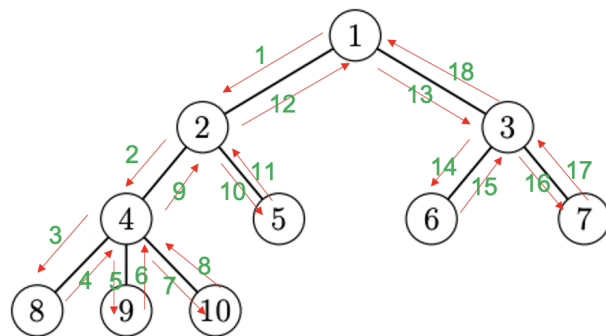
1	2	4	8	4	9	4	10	4	2	5	2	1	3	6	3	7	3	1
---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---

Algorithm 3: Doubling Trees

- 1: **Input:** complete graph G with costs C
- 2: $T \leftarrow$ MST of G
- 3: $T' \leftarrow$ double every edge of T
- 4: $\mathcal{E} \leftarrow$ Euler tour of T'
- 5: **return** TSP-tour from $\mathcal{E}$ by **shortcutting**

Theorem 13 Doubling Trees is a 2-approximation for $\triangle$ TSP.

Shortcutting: traverse $\mathcal{E}$; keep only the **first appearance** of each vertex. By the $\triangle$ -inequality, shortcuts never increase cost.



Euler tour:

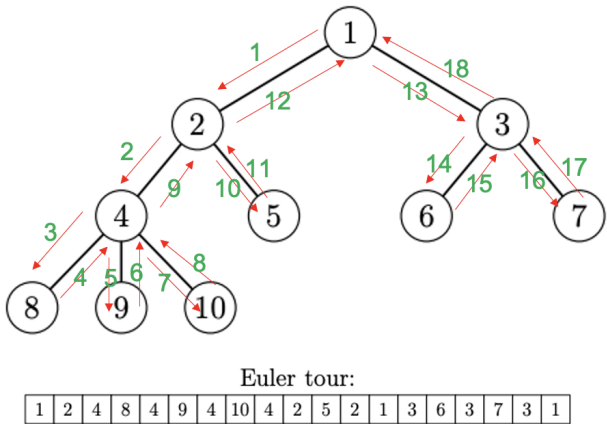
1	2	4	8	4	9	4	10	4	2	5	2	1	3	6	3	7	3	1
---	---	---	---	---	---	---	----	---	---	---	---	---	---	---	---	---	---	---

3.2 The Algorithm and Proof

Algorithm 4: Doubling Trees

- 1: **Input:** complete graph G with costs C
- 2: $T \leftarrow$ MST of G
- 3: $T' \leftarrow$ double every edge of T
- 4: $\mathcal{E} \leftarrow$ Euler tour of T'
- 5: **return** TSP-tour from $\mathcal{E}$ by **shortcutting**

Shortcutting: traverse $\mathcal{E}$; keep only the **first appearance** of each vertex. By the $\triangle$ -inequality, shortcuts never increase cost.



Theorem 14 Doubling Trees is a 2-approximation for $\triangle$ TSP.

Proof. Let L = output tour, T = MST, T' = doubled tree.

$$c(T') = 2c(T) \leq 2 \cdot \text{OPT}$$

since $\text{OPT} \geq \text{MST cost}$ (Key Observation).

Shortcutting via $\triangle$ -ineq. gives $c(L) \leq c(\mathcal{E}) = c(T')$, so:

$$c(L) \leq c(T') \leq 2 \cdot \text{OPT}. \quad \square$$

4. Christofides Algorithm

Doubling the whole MST is **wasteful** — we only need to fix the **odd-degree vertices** to make the graph Eulerian.

Doubling the whole MST is **wasteful** — we only need to fix the **odd-degree vertices** to make the graph Eulerian.

Lemma 17 In any graph, the number of **odd-degree vertices** is **even**.

Doubling the whole MST is **wasteful** — we only need to fix the **odd-degree vertices** to make the graph Eulerian.

Lemma 19 In any graph, the number of **odd-degree vertices** is **even**.

Lemma 20 Let G be a connected graph and M a **perfect matching** on the odd-degree vertices of G . Then $G + M$ is **Eulerian**.

Proof. Adding M makes every odd-degree vertex even-degree; all others remain even. $\square$

Doubling the whole MST is **wasteful** — we only need to fix the **odd-degree vertices** to make the graph Eulerian.

Lemma 21 In any graph, the number of **odd-degree vertices** is **even**.

Lemma 22 Let G be a connected graph and M a **perfect matching** on the odd-degree vertices of G . Then $G + M$ is **Eulerian**.

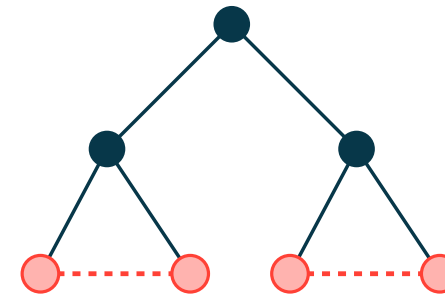
Proof. Adding M makes every odd-degree vertex even-degree; all others remain even. $\square$

Remark

A minimum-cost perfect matching on m vertices can be found in polynomial time.

Algorithm 5: Christofides

- 1: **Input:** complete graph G with costs C
 - 2: $T \leftarrow$ MST of G
 - 3: $O \leftarrow$ odd-degree vertices of T
 - 4: $M \leftarrow$ min-cost perfect matching in $G[O]$
 - 5: $H \leftarrow T + M$ (multigraph)
 - 6: $\mathcal{E} \leftarrow$ Euler tour of H
 - 7: **return** TSP-tour from $\mathcal{E}$ by shortcutting
-



Red = odd-degree; dashed = matching M

Theorem 23 The Christofides algorithm is a $\frac{3}{2}$ -**approximation** for $\triangle\text{TSP}$.

Theorem 25 The Christofides algorithm is a $\frac{3}{2}$ -**approximation** for $\triangle$ TSP.

We need the following key lemma:

Lemma 26 Let OPT be the optimal TSP cost, $W \subseteq V(G)$ with $|W|$ even, and M a min-cost perfect matching in $G[W]$. Then

$$c(M) \leq \frac{\text{OPT}}{2}.$$

Theorem 27 The Christofides algorithm is a $\frac{3}{2}$ -**approximation** for $\triangle$ TSP.

We need the following key lemma:

Lemma 28 Let OPT be the optimal TSP cost, $W \subseteq V(G)$ with $|W|$ even, and M a min-cost perfect matching in $G[W]$. Then

$$c(M) \leq \frac{\text{OPT}}{2}.$$

Proof of the theorem (given the lemma). Let L = Christofides output, T = MST.

$$c(L) \leq c(T) + c(M) \leq \text{OPT} + \frac{\text{OPT}}{2} = \frac{3}{2} \cdot \text{OPT}. \quad \square$$

The first inequality: shortcutting from the Euler tour of $T + M$ doesn't increase cost. The second: $c(T) \leq \text{OPT}$ (Key Observation) and $c(M) \leq \frac{\text{OPT}}{2}$ (lemma, with $W = O$, $|O|$ even by the odd-degree lemma).

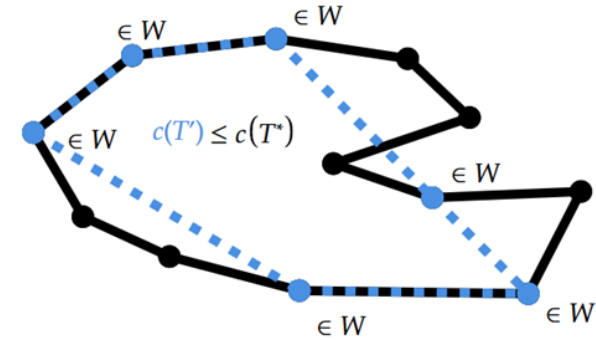
Lemma 29 $c(M) \leq \frac{\text{OPT}}{2}$ for any $W \subseteq V$ with $|W|$ even, where M is a min-cost perfect matching in $G[W]$.

Lemma 30 $c(M) \leq \frac{\text{OPT}}{2}$ for any $W \subseteq V$ with $|W|$ even, where M is a min-cost perfect matching in $G[W]$.

Let T^* = optimal TSP-tour (cost OPT). Let T' = the tour in $G[W]$ obtained by **shortcutting** T^* to only the vertices of W .

By the $\triangle$ -inequality, shortcutting never increases cost:

$$c(T') \leq c(T^*) = \text{OPT}.$$



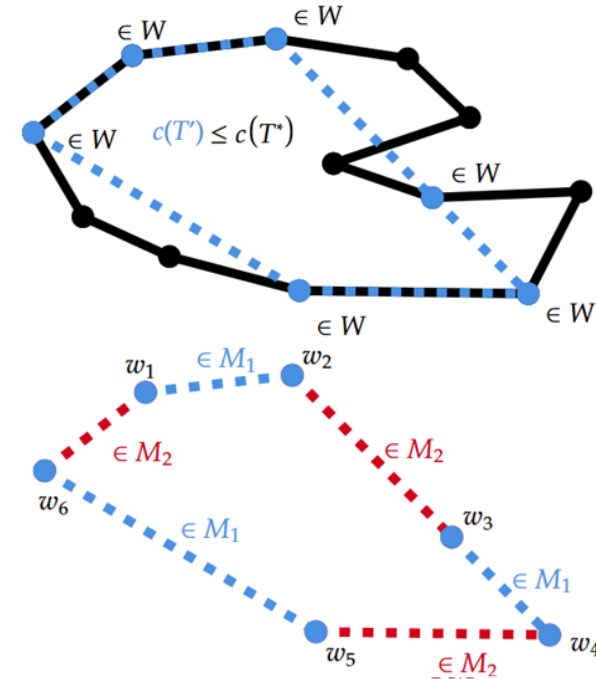
Lemma 31 $c(M) \leq \frac{\text{OPT}}{2}$ for any $W \subseteq V$ with $|W|$ even, where M is a min-cost perfect matching in $G[W]$.

Let T^* = optimal TSP-tour (cost OPT). Let T' = the tour in $G[W]$ obtained by **shortcutting** T^* to only the vertices of W .

By the $\triangle$ -inequality, shortcutting never increases cost:

$$c(T') \leq c(T^*) = \text{OPT}.$$

Since $|W|$ is **even**, T' is an **edge-disjoint union of two perfect matchings** M_1, M_2 on W .



4.4 Proof of Key Lemma

Lemma 32 $c(M) \leq \frac{\text{OPT}}{2}$ for any $W \subseteq V$ with $|W|$ even, where M is a min-cost perfect matching in $G[W]$.

Let T^* = optimal TSP-tour (cost OPT). Let T' = the tour in $G[W]$ obtained by **shortcutting** T^* to only the vertices of W .

By the $\triangle$ -inequality, shortcutting never increases cost:

$$c(T') \leq c(T^*) = \text{OPT}.$$

Since $|W|$ is **even**, T' is an **edge-disjoint union of two perfect matchings** M_1, M_2 on W .

Since M is the **minimum-cost** perfect matching in $G[W]$:

$$c(M) \leq \min(c(M_1), c(M_2)) \leq \frac{c(M_1) + c(M_2)}{2} = \frac{c(T')}{2} \leq \frac{\text{OPT}}{2}. \quad \square$$

